CS604 COMPUTER VISION · TEAM 06

Your Questions, Answered in Plain English

The linear-probe head and the prototype head, the two training steps, LoRA capacity, flip TTA, and why LoRA gets worse with longer training. Plus questions the instructor may ask.

Based on the slides `cs604_team06_slides_v2.pptx`, their speaker notes and the report. All numbers come from these files.

The whole idea in four sentences

The head is the last layer of the model: 10,000 "reference cards", one per class. Before the main training, we must decide what to write on these cards. The linear-probe head is trained on the training photos first, so it starts high (62.1%) but makes the model memorize and ends 2.1 to 3.1 points lower. The prototype head is just the average look of each class, with no training, so it starts lower (44.0%) but ends higher.

Contents

Part 1: Your questions **2**

Q1. The linear-probe head and the prototype head . . . 2
Q2. What "the head has seen the answers" means . . . 4
Q3. Were the photos already trained on before? . . . 5
Q4. Which two steps? . . . 5
Q5. Our final model's two steps . . . 6
Q6. From 768 numbers to 10,000 scores . . . 7
Q7. "Does LoRA lack capacity?" . . . 7
Q8. LP-FT and the linear-probe head . . . 8
Q9. Flip TTA . . . 9
Q10. "Weak, then wrong signal" . . . 10
Q11. Why LoRA gets worse with longer training . . . 11

Part 2: Questions the instructor may ask **12**

Key numbers for these topics **16**

Part 1: Your questions

Q1. What are the linear-probe head and the prototype head, and how are they different?

First: what is a head?

The last layer of the model is called the **head**. Its job is to give each of the 10,000 classes a score. Think of it as 10,000 **reference cards**, one card per class. A photo first goes through the backbone and becomes 768 numbers (its **features**). Then these numbers are compared with every card. The more the photo matches a card, the higher that class scores.

We changed to a new task, so we need new cards. The question is: **before the main training starts, what do we write on the cards?** The linear-probe head and the prototype head are two different answers.

The linear-probe head: cards made by "practicing" first

- **How:** we lock the backbone so it cannot change. Then we train only the cards, on the training photos, until they classify most of those photos correctly.
- **Result:** 62.1% before the main training. A very high start.
- **Problem:** the cards were practiced on **the same training photos** that the main training uses. So the head has "seen the answers" already:
  - When the main training starts, the model thinks "I already know this". The starting loss is only 2.29, so the push to learn is weak.
  - Once the backbone starts to change, the cards no longer fit the new features. With LoRA, the loss even **went up** in the first epoch, from 2.22 to 2.69.
  - In the end, the model memorizes the 45 photos per class.

The prototype head: the "average look" of each class

- **How:** no training at all. For each class, we average the features of its 45 photos. This gives the "typical look" of the class. We multiply it by 40 and write it on the card.
- **Scoring:** how similar the photo is to the class's average look (the cosine similarity), times 40.
- **Result:** only 44.0% before training, 18 points lower than the linear-probe head. But it was never tuned to any single photo, so there is nothing to memorize. The starting loss is 4.39: the model knows it still has a lot to learn, so it learns properly.

	Linear-probe head	Prototype head
How it is made	trained on the training photos	average of each class, no training
Before the main training	62.1%	44.0%
Starting loss	2.29 (“I already know this”)	4.39 (“I still have a lot to learn”)
After training	2.1–3.1 points lower	our choice
Example: full fine-tuning, 3 epochs (holdout)	70.88%	73.38%
Example: final recipe (validation)	73.10%	76.22%

Everyday analogy

Two students prepare to recognize 10,000 kinds of birds, with 45 practice photos each. Student A memorizes the answers to the practice photos. A's first mock exam score is very high, but A stops really learning and relies on the memorized answers. On new photos, A does worse. (The linear-probe head.) Student B only remembers what each kind of bird "roughly looks like". B's first mock exam score is lower, but B never memorized anything and keeps learning the real patterns. In the real exam, B does better. (The prototype head.)

The prototype head is not magic

We also tried a random head, which knows nothing at the start. It ended level with the prototype head (71.95% vs. 72.00%). So the real problem is that the linear-probe head was **practiced on the training photos first**. The prototype head simply avoids that, and it starts better and learns faster than a random head.

Do not mix these up

The “linear probe” on slide 16 is a **method**: only the head is ever trained (61.94%). The “linear-probe head” on slide 6 is a **starting point**: practice the head first, then fine-tune the whole model.

Answer you can say

The linear-probe head is trained on our training photos before fine-tuning, so it starts high but leads the model to memorize. The prototype head is just the average feature of each class, with no training, so it starts lower but ends 2 to 3 points higher.

Q2. What does “the head has seen the answers” mean?

It is **not** cheating. The model never saw the validation or test photos. It means: **the practice questions of the main training were already practiced by the head.**

Two steps, with the same photos

1. **Step 1 (prepare the head):** the backbone stays fixed. We use the training photos and their labels to train only the cards, until they classify most of these photos correctly.
2. **Step 2 (the main training):** we train the whole model on **the same training photos, with the same labels**.

The “practice questions” of step 2 are exactly the ones the head already mastered in step 1.

Why this is bad

A model learns by fixing mistakes. More mistakes mean a bigger loss, and a stronger signal telling the backbone how to change.

- **Almost no mistakes at the start:** the head already gets most training photos right. The starting loss is only 2.29, and the backbone gets almost no push to change. But our task needs the features to change, to tell apart very small differences.
- **The remaining signal comes from a few wrong photos:** the model focuses on those few photos, which is just memorizing them.
- **The answers “expire”:** the cards were tuned to the old features. When the backbone starts to change, the cards no longer fit. So with LoRA the loss went up from 2.22 to 2.69 in the first epoch, and the model had to learn the photos all over again.
- **The result:** a lower training loss (1.77 vs. 1.85), because it memorized well; but a lower validation score (73.10% vs. 76.22%), because it fails on new photos.

The prototype head also uses the training photos. Why is it different?

The difference is whether the head practiced “getting each photo right”:

- **Linear-probe head:** adjusted again and again until each training photo is right. It even learned the quirks of single photos. Like memorizing an answer key.
- **Prototype head:** only averages the features of each class. It never checks whether each photo is right, and it takes no training steps. Like reading a summary.

So for the prototype head, the training photos are still “new questions”: the starting loss is 4.39, the model makes mistakes, and it learns from them.

Everyday analogy

Before a final exam, the teacher gives 45 practice questions per chapter, and the review class uses only these questions. Student A memorized the answers before the review class, so A thinks “I know all of this”, skips the understanding, and crams whatever is still wrong. On new exam questions, A fails. Student B only read the summary notes, gets some practice questions wrong in class, and learns the ideas behind them. On new questions, B does better.

Evidence: a random head, which has not seen any answers, ends level with the prototype head (71.95% vs. 72.00%). So the problem is not “prototypes or not”, but whether the head **practiced on the training**

photos first.

Q3. So the photos we train on were already trained on before?

Yes, roughly. But to be exact: **only the head was trained on these photos before, not the whole model.**

- **Step 1:** the backbone is locked. Only the head practiced on these photos and can already get most of them right.
- **Step 2:** the whole model trains on the same photos. For the head, they are "old questions". For the backbone, they are "new questions".

The problem is not using the photos twice

Normal training sees the same photos many times anyway: full fine-tuning runs 8 epochs, so each photo is seen 8 times. The real problem is that **the head learned first and used up the learning signal**:

- The model only gets a "how to change" signal when it gets something wrong. The head already gets most photos right, so the loss is small (2.29) and the signal is weak.
- The part that really needs to learn is the backbone, which must learn to see small differences. It gets too little signal to learn.
- As soon as the backbone changes a little, the head's memorized answers stop fitting, and the model has to memorize the photos again.

Everyday analogy

Two partners solve problems together. The backbone understands the question; the head writes the answer. The teacher only explains the mistakes. With the linear-probe head, the head first memorized the answers to these problems alone. When the two work together, most answers are already right, the teacher has almost nothing to explain, and the backbone never hears an explanation. Its understanding does not improve. With the prototype head, the head only knows roughly what each class looks like and still makes many mistakes. The teacher explains a lot, and the backbone really learns.

Q4. Which two steps?

The model has two parts

Photo → **backbone** (85.8 million parameters; it "understands" the photo) → 768 numbers → **head** (10,000 cards) → 10,000 scores.

The pretrained model came with an old head that only knew ImageNet's 1,000 classes. We throw it away and put in a new 10,000-class head. When the new head is put in, its cards are still blank.

Step 1: fill in the cards of the new head (before the main training)

There are two ways to fill them:

- **Linear-probe head:**
  1. lock the backbone, so none of its parameters can change;
  2. run all 450,000 training photos through the backbone, so each photo becomes 768 numbers;
  3. train only the head: adjust the numbers on the cards until they name the right class for each

training photo.

After this, the head already gets most training photos right.

- **Prototype head:** also compute the 768 numbers of each photo, but **do not train**. Average the numbers of each class's 45 photos and write them straight onto that class's card. Done.

Step 2: the main training

This is the full fine-tuning (8 epochs, 1.03 GPU-hours) or LoRA (3 epochs, 0.54 GPU-hours) on the slides. We unlock the backbone and train the backbone and the head together, on the same 450,000 training photos.

	Step 1: fill in the cards	**Step 2: main training**
Linear-probe head	**train** the head on the training photos, backbone locked	train the whole model on the same photos
Prototype head	**average** the training photos, no training	exactly the same

Key point

The two ways differ **only in step 1**. Step 2 is the same. On slide 7, "Extract features" and "Prototype head" are step 1, and "Fine-tune" is step 2.

Q5. Did our final full fine-tuning model also go through these two steps?

Yes, exactly.

Step 1: fill in the head (prototype head), no training

1. Lock the backbone and run the 450,000 training photos through it once. Each photo becomes 768 numbers. This takes about 6 minutes.
2. Average each class, multiply by 40, and write it onto that class's card.

No training happens here. Not a single backbone parameter changes.

Step 2: the main training (full fine-tuning)

- Unlock the backbone. All 85.8 million backbone parameters and the 7.69 million head parameters train together.
- 8 epochs, on all 45 photos per class, 14,056 steps in total.

The budget: the 6 minutes of step 1 also count as GPU time. The total of 1.03 GPU-hours includes them.

On the slides: on slide 7, "Extract features" and "Prototype head" are step 1, "Fine-tune" is step 2, and "Predict" (score each photo and its mirror image, then add) comes after training. The card on slide 10 that says "Head starts from class prototypes $\times$ 40" describes step 1.

The final LoRA model works the same way. Step 1 is the same prototype head. In step 2, the backbone stays frozen, and only the small LoRA matrices and the head train, for 3 epochs.

Q6. How do the 768 numbers turn into 10,000 class scores?

Through the 10,000 "cards". Each card also holds 768 numbers, one for each of the photo's 768 numbers.

How a score is computed

Multiply the photo's 768 numbers with one card's 768 numbers, pair by pair, and add everything up (plus one small extra number, the bias). The result is one score: how much the photo looks like this class. Do this for all 10,000 cards and you get 10,000 scores. **The class with the highest score is the model's answer.**

A small example: 3 numbers instead of 768, and 3 classes instead of 10,000

Say a photo becomes (0.9, 0.1, 0.4). Think of them roughly as "how feathery, how green, how pointy the beak is". (In the real model the numbers are learned, and humans cannot name what each one means.)

Card	Numbers on the card	Calculation	Score
Sparrow	(1.0, 0.0, 0.5)	$0.9 \times 1.0 + 0.1 \times 0.0 + 0.4 \times 0.5$	**1.10**
Crow	(0.0, 1.0, 0.0)	$0.9 \times 0 + 0.1 \times 1.0 + 0.4 \times 0$	0.10
Eagle	(0.2, 0.3, 1.0)	$0.9 \times 0.2 + 0.1 \times 0.3 + 0.4 \times 1.0$	0.61

The sparrow scores highest, so the model answers "sparrow". The real model does exactly this, with 768 numbers and 10,000 cards.

This is where the head's size comes from: 10,000 cards $\times$ 768 numbers $=$ 7,680,000, plus one bias per class (10,000), gives 7,690,000 parameters. That is the "Head: 768 $\rightarrow$ 10,000, 7.69M weights" on slide 10.

How it connects:

- **Prototype head:** each card holds the class's "average numbers" (times 40). The more a photo looks like a class's average, the higher its score. That is slide 7's "each logit $=$ 40$\times$ cosine(photo, class mean)".
- **Training:** keeps adjusting the numbers on the cards (and, in full fine-tuning, the backbone too), so the right class of each photo scores highest.

Q7. What does "Does LoRA lack capacity?" mean?

It is one row of the timeline on slide 4:

Oct 18 Does LoRA lack capacity? Also put LoRA on proj, fc1 and fc2 Worked: +0.88

It asks: **does LoRA learn badly because it can change too few things?**

What "capacity" means

Capacity is how much a model is able to change and learn. The more adjustable parameters it has, and the more places they sit in, the more it can learn.

Why we suspected it

At first, we followed the reference in the project brief and put LoRA only on the **qkv** layer of each block, the entrance of the attention part. But each block has 4 linear layers:

- **qkv:** the entrance of attention;

- **proj:** the exit of attention;
- **fc1 and fc2:** the two layers of the MLP (the "thinking" part).

With LoRA only on qkv, each block can change in only one place, and the MLP cannot change at all. Yet the **MLP holds about two thirds of each block's weights** (fc1 and fc2 have about 4.72 million of the block's 7.09 million). To tell apart 10,000 very similar classes, that may not be enough room to change.

How we tested it

We also put LoRA on proj, fc1 and fc2, and kept everything else the same:

	Where LoRA sits	LoRA parameters	Share of backbone	Holdout
Before	qkv only	294,912	0.34%	68.57%
After	qkv, proj, fc1, fc2	1,179,648	1.375%	69.45% (+0.88)

What we concluded

Capacity was a bit too small: opening up the MLP added about 0.9 points. So our final LoRA model uses all four layers. It still trains only 1.375% of the backbone, far below the 5% example in the brief.

Its place in the story

This is the second guess in slide 4's "detective work":

1. First guess, "the learning rate is too high": failed (-0.12).
2. Second guess, "too little capacity": helped, but only $+0.88$.
3. Third guess, "the head is the problem": the big one ($+2.5$).

Everyday analogy

You renovate a house, but you may only change the doors (qkv). That limits what you can do. If you may also change the windows and the furniture (proj, fc1, fc2), you can really adapt the house to its new use.

Q8. Is LP-FT the same as the linear-probe head approach?

Yes. LP-FT is exactly the approach that uses a linear-probe head. Its name gives the two steps:

- **LP** (Linear Probe): step 1. Freeze the backbone and train only the head. The result is the "linear-probe head".
- **FT** (Fine-Tune): step 2. Start from this head and train the whole model.

	Step 1: fill in the head	Step 2
LP-FT (recommended in the paper)	linear probe: freeze the backbone, train the head on the training photos	fine-tune the whole model
Ours	prototype: average each class's features $\times$ 40, no training	fine-tune the whole model

"Linear" is not the difference

All our heads have the same structure: one linear layer, that is, 10,000 cards of 768 numbers. The random head, the linear-probe head and the prototype head look exactly alike. **Only the starting numbers on the cards differ**: random numbers; numbers trained on the training photos; or class averages. So the precise name is "linear-probe head", not "linear head".

Where the word "probe" comes from: it was first used to "probe" frozen features. You train a simple linear classifier on top of features that do not change, to test how useful they are. Our backbone scores 61.94% this way (slide 16).

Why the paper recommends LP-FT: Kumar and colleagues argue that a random head makes large errors at the start of training, and these can "bend" the good pretrained features. So they first train a good head and then fine-tune. This protects the original features and makes the model more stable on other kinds of data.

Why it does not help here:

- Our task needs the features to change a lot, to tell apart 10,000 very similar classes.
- The linear-probe head was practiced on the same few photos (45 per class) used later, so the model memorizes.

In all three methods we tested, the linear-probe head ended 2.1 to 3.1 points lower. And a random head ended level with the prototype head, so here the random head did not cause the harm the paper worries about.

Q9. What is flip TTA?

It is a trick used at **prediction time**, after training, when the model answers for new photos. **TTA** means test-time augmentation: make a small change to the photo at prediction time, so the model gets another look. **Flip** means a left-right mirror image.

How it works

1. The original photo goes into the model and gets 10,000 scores.
2. The mirror image goes into the model and gets another 10,000 scores.
3. Add the two sets of scores, class by class.
4. The class with the highest total is the answer.

Why this is allowed

A bird facing left and a bird facing right are still the same kind of bird. Mirroring never changes the class. We also flipped photos at random during training (the "flip" on slide 10), so the model knows both directions.

Why it helps

One look can be fooled by a chance detail. Two looks combined work like a second opinion, so some chance mistakes cancel out.

An example (made-up numbers, just to show the idea)

	Sparrow	Finch	Crow	Answer on its own
Original photo	5.0	5.2	1.0	finch (wrong)
Mirror image	5.6	4.9	1.2	sparrow
Both added	**10.6**	10.1	2.2	**sparrow (right)**

What it did in our project

- $+0.27$ points on the validation set (72.57% $\rightarrow$ 72.84%); it changed 4.6% of the predictions.
- The cost: the model runs twice per photo, so it takes about twice as long. That is why slide 12 shows "14.95 ms (2 passes)" for full fine-tuning.

TTA does not change any parameter; it only runs the model one more time. Only the final full fine-tuning model uses it. The LoRA model does not: its selling point is speed (7.61 ms after merging), and TTA would make it slower.

Q10. "Weak, then wrong signal": what does it mean in plain words?

This is step 2 of the explanation on slide 8. The linear-probe head causes two problems: first the signal is **too weak**, then it points **the wrong way**.

Problem 1, "weak": almost no mistakes, so almost nothing to learn

A model learns by fixing mistakes. More mistakes mean a bigger loss and a stronger signal telling the backbone how to change. The linear-probe head already practiced on these training photos and gets most of them right, so the starting loss is only about 2.3. Few mistakes means a weak signal: the backbone hardly knows which way to change.

Problem 2, "wrong": once the backbone moves, the head stops fitting

The 10,000 cards of the linear-probe head were tuned precisely to the 768 numbers of the **old** backbone. When the main training starts, the backbone (here, the parts with LoRA) begins to change, so the 768 numbers of every photo change too. The cards are still tuned to the old numbers, so they no longer fit: photos that were right become wrong.

So instead of falling, the training loss **rose** in the first epoch, from about 2.2 to about 2.7. In normal training the loss should keep falling, so a rising loss is a warning sign. It was the first odd thing we noticed (slide 4: "its loss rose in epoch 1").

At this point the model is busy making the head fit the moving features again. That is memorizing the training photos a second time, not learning skills that work on new photos.

Everyday analogy

The linear-probe head is a suit tailored exactly to your old body shape. When you start working out (training), you think "the suit fits fine, no need to change", so you hardly train (a weak signal). Then your body changes (the backbone changes), the suit stops fitting (the loss rises), and you keep altering the suit instead of focusing on your body. The prototype head is a loose one-size shirt. It does not fit well from the start (loss about 4.4, many mistakes), which tells you to train properly. It was never tailored, so a changing body cannot suddenly make it "not fit".

Q11. Why does full fine-tuning get better with longer training, while LoRA gets worse?

What we saw

- **Full fine-tuning:** from 3 to 8 epochs, $+2.2$ **points**.
- **LoRA:** from 3 to 6 epochs, -0.46 **points**. The training loss kept falling (1.95 $\rightarrow$ 1.70), but the holdout score fell too: it was memorizing (overfitting).

This is strange: LoRA trains far fewer parameters than full fine-tuning, so it should memorize less, not more.

Our guess: most of what LoRA trains is actually the head

"LoRA trains only 1.375%" counts only the small LoRA matrices (1.18 million parameters). But the **head also trains** in LoRA, and the head has 7.69 million parameters. Together LoRA trains 8.87 million parameters, and **87% of them are the head**. So "training LoRA" is mostly training the head.

The two recipes treat the same head very differently

Setting	Full fine-tuning	LoRA	In plain words
Learning rate	0.0001	0.001 (**10 times**)	how big each change is; LoRA's head changes fast and can quickly chase every training photo
Weight decay	0.05	0.0001 (**1/500**)	a "brake" that keeps the numbers from getting extreme; LoRA has almost no brake
Drop path	0.1	none	randomly skips some layers in training, adding noise so the model cannot memorize details; LoRA has no such protection

So in LoRA, the head changes fast, with no brake and no noise. Given more time, it memorizes the 43 training photos of each class and gets worse on new photos. In full fine-tuning, the head changes slowly, with a firm brake and some noise. It does not memorize easily, so more training keeps helping.

Everyday analogy

Two students study a few extra days. Student A (full fine-tuning) has a strict tutor who makes them go slowly, corrects them often and adds some surprise questions. The extra days make A's knowledge more solid. Student B (LoRA) spends most of the time alone, racing through the same practice set with nobody watching. After the extra days, B knows that practice set by heart, and does worse on new questions.

This is only a guess; we have not tested it

We prepared a notebook to test it (`methods/10_lora_head_lr.ipynb`): train LoRA for 6 epochs and lower only the head's learning rate to 0.0001, the same as in full fine-tuning, with everything else the same.

- If the 6-epoch run no longer drops (back to about 71.98% or higher), the head was memorizing.
- If it still drops, the problem is somewhere else.

Part 2: Questions the instructor may ask

These questions are about the same topics as Part 1. Each answer is short enough to say out loud. Where we did not run an experiment, the answer says so; never guess beyond what we tested.

Q1. LP-FT is a recommended recipe. Why did the linear-probe head hurt in your project?

Speaker: C

Answer you can say

LP-FT is designed to protect the pretrained features. Our task needs the features to change a lot, because the 10,000 classes are very similar. Also, our linear-probe head was fitted on the same 45 photos per class that we then fine-tune on. So it starts with a low loss and a weak learning signal, and the model ends up memorizing. In all three methods we tested, it ended 2.1 to 3.1 points lower.

Possible follow-up questions

Follow-up: So is the LP-FT paper wrong?
Answer: No. It aims at robustness on new kinds of data. In our setting, with many classes and few photos per class, its starting point does not help.

Q2. Is the prototype head your own invention?

Speaker: C

Answer you can say

No. It is weight imprinting, from Qi and colleagues at CVPR 2018. What is new is our finding: in our setting, it beats the recommended LP-FT start for all three methods we tested.

Q3. If a random head works as well as the prototype head, why use prototypes at all?

Speaker: C

Answer you can say

The random head ties at the end, 71.95 against 72.00. That tells us the harm comes from fitting the head on the training photos. But the prototype head starts at 44% instead of chance, and it learns faster: after the first epoch it was at 63.3%, against 58.4% for the random head. And it costs only about six minutes.

Q4. How did you choose the factor 40?

Speaker: C

Answer you can say

We tried 5, 10, 15, 20, 30, 40, 60 and 80, and picked the value with the lowest cross-entropy on 10,000 training features. That was 40. No gradient steps are involved.

Possible follow-up questions

Follow-up: Why multiply at all?
Answer: The cosine similarity is only between -1 and 1, so without a factor the softmax would give almost the same probability to every class. Multiplying by 40 spreads the scores apart.

Q5. Why is a higher starting loss a good thing?

Speaker: C

Answer you can say

A high loss means the model still makes many mistakes on the training photos, so the learning signal is strong and the backbone is pushed to adapt. The linear-probe head starts at 2.29 because it has already fitted those photos, so its signal is weak. The prototype head starts at 4.39. A random head starts at 9.28, close to pure guessing.

Possible follow-up questions

Follow-up: Can you turn 4.39 into a probability?
Answer: Not directly. It is a training loss with label smoothing and data augmentation, so we only compare the three values with each other.

Q6. What does the head actually compute?

Speaker: B

Answer you can say

For each class, the head multiplies the photo's 768 feature numbers with that class's 768 weights and adds them up, plus a bias. That gives 10,000 scores, and the highest one is the prediction. So the head has 768 times 10,000 weights plus 10,000 biases: 7.69 million parameters.

Q7. Why did LoRA's training loss go up in the first epoch?

Speaker: B or C

Answer you can say

That run used the linear-probe head, which was fitted to the old features. When LoRA starts changing the features, the head no longer matches them, so photos it got right become wrong. The loss rose from 2.22 to 2.69 in the first epoch. That was the first warning sign that led us to look at the head.

Possible follow-up questions

Follow-up: Why did you first suspect the learning rate?
Answer: A rising loss often means the steps are too big. But halving the learning rate changed nothing (-0.12), so we ruled it out.

Q8. Adding LoRA to more layers adds parameters. Is it still parameter-efficient?

Speaker: B

Answer you can say

Yes. Going from qkv only to all four linear layers raised LoRA from 0.34% to 1.375% of the backbone. That is still far below the 5% example in the brief. It added 0.88 points, because two thirds of each block's weights are in the MLP.

Possible follow-up questions

Follow-up: Did you try other ranks?
Answer: No. We kept rank 8, as in the brief, and added capacity by placing LoRA on more layers instead.

Q9. Why do you use flip TTA only for full fine-tuning?

Speaker: D

Answer you can say

TTA runs the model twice per photo, so it doubles the time: 14.95 milliseconds for full fine-tuning. LoRA's selling point is that it is as fast as the base model after merging, 7.61 milliseconds, and TTA would double that. For full fine-tuning, TTA added 0.27 points on validation.

Q10. Is the 0.27-point gain from TTA real?

Speaker: D

Answer you can say

It is at the edge of our noise level, which is about 0.3 points. But it changed 4.6% of the predictions, and it is a cheap, common trick, so we kept it for the full fine-tuning model.

Q11. Why would LoRA overfit, when it trains fewer parameters than full fine-tuning?

Speaker: D

Answer you can say

Most of what LoRA trains is actually the head: 7.69 million of its 8.87 million trainable weights. In our LoRA recipe the head gets a 10 times higher learning rate than in full fine-tuning, almost no weight decay, and no drop path. So we think the head memorizes the photos. We have not tested this yet. The test is a 6-epoch LoRA run with only the head's learning rate lowered to 1e-4.

Q12. How do you know the problem is memorization, and not something else?

Speaker: C

Answer you can say

Two pieces of evidence. With the linear-probe head, the training loss is lower, 1.77 against 1.85, but the validation accuracy is lower, 73.10 against 76.22. And a random head, which was never fitted to the training photos, ends level with the prototype head. Our explanation fits the loss curves and the control, but it is not a full proof.

Q13. Does building the prototype head count toward your GPU budget?

Speaker: A

Answer you can say

Yes. Extracting the features takes about six minutes, and it is included in our GPU-hours: 1.03 hours for full fine-tuning and 0.54 for LoRA, both under the 1.2-hour budget.

Q14. Would the linear-probe head still hurt if it were trained on different photos?

Speaker: C

Answer you can say

We did not test that. Our explanation says the harm comes from fitting the head on the same photos we then fine-tune on, so a head fitted on other photos might behave differently. It would be a good next experiment.

Q15. What do the 768 numbers mean?

Speaker: B

Answer you can say

They are features the model learned during pretraining. Each number captures some visual pattern, but single numbers are hard for humans to name. Together they describe what is in the photo, and the [CLS] token collects them into one summary for the head.

Key numbers for these topics

Item	Number
Head size	$768 \times 10{,}000 + 10{,}000 = 7{,}690{,}000$ parameters
Before the main training (holdout)	linear-probe head 62.1%; prototype head 44.0%
Starting loss (first 100 LoRA steps)	linear-probe 2.29; prototype 4.39; random 9.28 (pure guessing: $\ln 10{,}000 \approx 9.21$)
LoRA loss with the linear-probe head, first epoch	rose from 2.22 to 2.69
After 3 epochs (holdout), linear-probe vs. prototype	full fine-tuning 70.88 vs. 73.38; LoRA 69.45 vs. 72.00; AdaptFormer 69.05 vs. 71.17
Final recipe (validation), linear-probe vs. prototype	73.10% vs. 76.22%; final training loss 1.77 vs. 1.85
Random head vs. prototype head (LoRA, holdout)	71.95% vs. 72.00%; after epoch 1: 58.4% vs. 63.3%
Feature extraction for the prototype head	about 6 minutes, counted in the GPU-hours
LoRA capacity	qkv only: 294,912 (0.34%), 68.57% $\rightarrow$ all four layers: 1,179,648 (1.375%), 69.45% ($+0.88$)
Flip TTA (validation)	$+0.27$ (72.57% $\rightarrow$ 72.84%); changes 4.6% of predictions; 14.95 ms per photo (2 passes)
Longer training	full fine-tuning 3 $\rightarrow$ 8 epochs: $+2.21$ (validation); LoRA 3 $\rightarrow$ 6 epochs: -0.46 (holdout; training loss 1.95 $\rightarrow$ 1.70)
What LoRA trains	1.18M LoRA weights + 7.69M head = 8.87M; 87% is the head
Head treatment, full fine-tuning vs. LoRA	learning rate 10^{-4} vs. 10^{-3}; weight decay 0.05 vs. 10^{-4}; drop path 0.1 vs. none